Business Analytics Assignment

AI Activity Report

Apify Web Scraper & NVIDIA NeMo Data Designer

DONE BY:
ADHIKKESH S K
CB.SC.U4CSE23101

Introduction

Artificial Intelligence is playing an increasingly important role in Business Analytics, particularly in the way data is collected, generated, and prepared for analysis. As part of the supplementary AI learning activity for this Business Analytics course, this report documents the exploration of two AI-driven platforms: Apify Web Scraper, used for automated web data extraction, and NVIDIA NeMo Data Designer, used for generating synthetic datasets using Large Language Models. The objective was to understand the working of each platform, perform a small hands-on demonstration, and evaluate their relevance to real-world Business Analytics use cases.

Learning Objectives

- To understand how automated web scraping tools extract structured data from websites.
- To understand how synthetic datasets can be generated using Large Language Models.
- To configure and run a working demonstration on both platforms.
- To evaluate the practical applications of both tools in Business Analytics.
- To identify the advantages and limitations observed while using each platform.

Activity 1: Apify Web Scraper

1.1 Introduction to Apify

Apify is a cloud-based web scraping and automation platform that allows structured data to be extracted from websites without installing any scraping software locally. The scraping logic is written and executed directly in the browser through a pre-built Actor called Web Scraper. Once a scraping run finishes, the extracted records are stored in an Apify Dataset, which can be exported in formats such as JSON, CSV, Excel, HTML Table, XML, or RSS.

1.2 Why Use Apify?

Problem	How Apify Solves It
Manual data collection is slow	Extracts data automatically from every matching page
Websites contain many pages	Follows links using a Link Selector and crawls automatically
Raw HTML is unstructured	Returns clean, structured JSON records ready for analysis
Modern websites load content dynamically	Runs a real headless browser that renders JavaScript
Data needs to be reused elsewhere	Exports directly to CSV, Excel, JSON or XML

1.3 Key Features of Apify Web Scraper

Feature	Description
Start URLs	The page(s) where the scraper begins crawling
Link Selector	CSS selector used to find and follow additional links

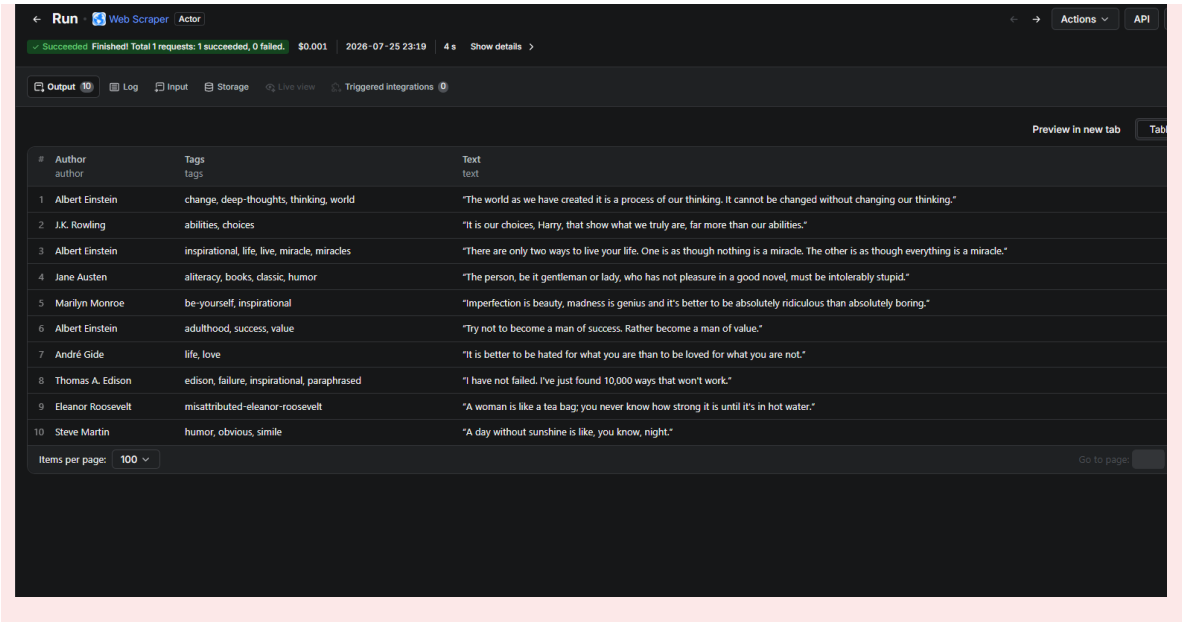
Page Function	Custom JavaScript that defines exactly what data to extract
Inject jQuery	Injects jQuery into the page so selectors can be written easily
Proxy Configuration	Rotates IP addresses to avoid being blocked
Dataset Export	Download results as JSON / CSV / Excel / XML / HTML Table

1.4 Activity Performed

An account was created on the Apify platform and the Web Scraper Actor was opened from the Apify Store. The practice website Quotes to Scrape (https://quotes.toscrape.com/) was chosen as the target because it is specifically designed for learning web scraping and does not restrict automated access.

The scraping task was configured with the following settings:

Setting	Value
Start URL	https://quotes.toscrape.com/
Inject jQuery	Enabled
Run Mode	Development
Maximum Requests	1
Link Selector	Empty (restricts crawl to the homepage only)
Wait Until	domcontentloaded



1.5 Page Function Used

The extraction logic was written inside the Page Function, which runs in the browser after the target page has fully loaded:

```
async function pageFunction(context) {

    const $ = context.jQuery;

    const quotes = [];



    $(".quote").each(function () {

        const tags = [];

        $(this).find(".tags .tag").each(function () {

            tags.push($(this).text());

        });



        quotes.push({

            text: $(this).find(".text").text(),

            author: $(this).find(".author").text(),

            tags: tags.join(", ")

        });

    });



    context.log.info(`Found ${quotes.length} quotes`);

    return quotes;

}
```

1.6 Working of the Code

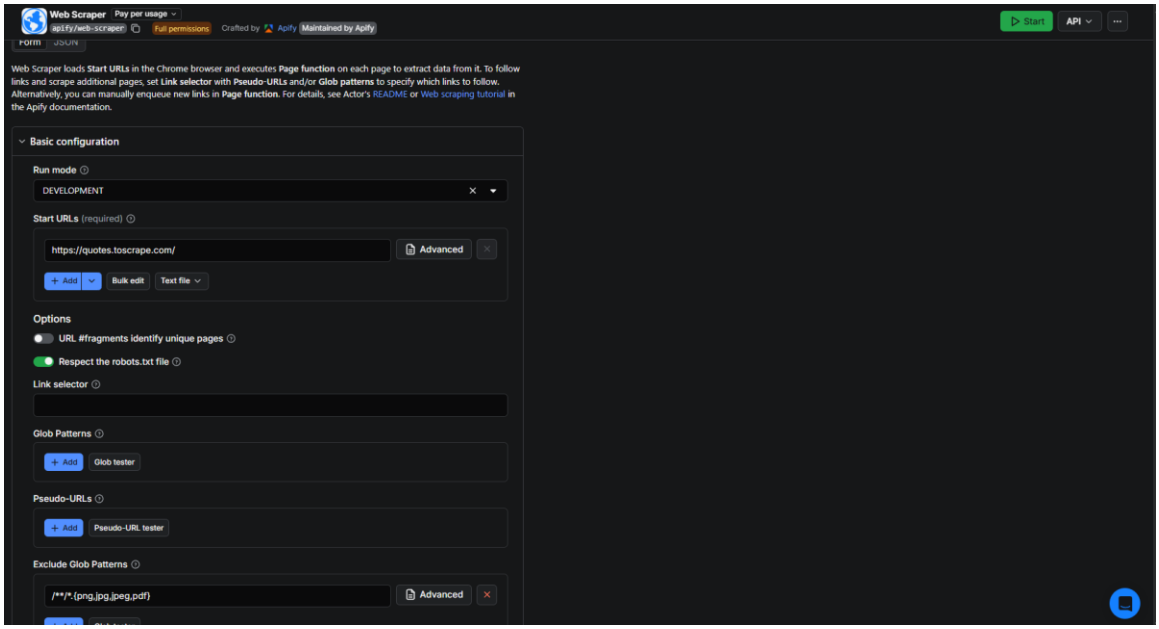
1. context.jQuery gives the Page Function access to the loaded page's HTML using familiar jQuery syntax.
2. The selector .quote matches every quote block displayed on the page.
3. For each quote block, .text extracts the quote itself and .author extracts the author's name.

4. A nested loop over .tags .tag collects every tag associated with that quote into an array, which is then joined into a comma-separated string.
5. Each extracted record is pushed into the quotes array as a JavaScript object.
6. The completed array is returned from the Page Function, and Apify automatically stores it as a Dataset.

1.7 Sample Output

After the run completed, the dataset contained structured records such as:


```
[
  {
    "text": "The world as we have created it is a process of our thinking.
It cannot be changed without changing our thinking.",
    "author": "Albert Einstein",
    "tags": "change, deep-thoughts, thinking, world"
  },
  {
    "text": "It is our choices, Harry, that show what we truly are,
far more than our abilities.",
    "author": "J.K. Rowling",
    "tags": "abilities, choices"
  }
]
```



1.8 Exporting the Dataset

Apify allows the collected dataset to be exported into several formats, which can be opened directly in Excel, Python, Power BI, or Tableau for further analysis.

Format	Typical Use
JSON	Programmatic use in Python / JavaScript
CSV	Quick analysis in Excel or Google Sheets
Excel (.xlsx)	Formatted spreadsheets with multiple fields
XML / RSS	Integration with other systems or feeds

1.9 Additional Exploration: Scraping a More Complex Website

To understand the limitations of simple CSS-selector scraping, the Web Scraper was also pointed at a larger, dynamically rendered website with anti-scraping protections in place.

1.10 Business Analytics Applications of Apify

Business Area	Example Use Case
E-commerce	Monitoring competitor product prices
Marketing	Collecting customer reviews for sentiment analysis
Tourism / Hospitality	Scraping hotel or restaurant ratings
Finance	Gathering stock-related news headlines
Research	Building custom datasets for academic analysis

Activity 2: NVIDIA NeMo Data Designer

2.1 Introduction

NVIDIA NeMo Data Designer is a synthetic data generation framework within the NVIDIA NeMo Platform. Rather than manually collecting and labelling real records, it lets a user define the structure of a dataset and have a Large Language Model generate realistic synthetic data that follows that structure. The workflow separates dataset design (schema, columns, rules) from dataset execution (which happens on the NeMo Platform).

2.2 Why Synthetic Data?

Challenge	Benefit of Synthetic Data
Limited real-world data	Generates additional records on demand
Sensitive / private customer data	Avoids exposing real personal information
Manual data collection is expensive	Reduces time and cost of dataset creation
Rare edge cases	Can simulate uncommon scenarios that are hard to collect naturally
Model training / evaluation	Produces large, labelled datasets quickly

2.3 Workflow of Data Designer

The synthetic data generation workflow followed in this activity was:

- Select an AI model and provider
- Create the model configuration
- Define the dataset schema (columns)
- Preview a small sample of generated records
- Generate the full dataset
- Download and evaluate the generated artifacts

2.4 Step 1 – Environment Setup

A Python virtual environment was created and the Data Designer SDK was installed. An NVIDIA API key was generated from the NVIDIA account portal and stored as an environment variable so it could be used by the SDK without hard-coding it into the script.

2.5 Step 2 – Configure the Model

The model configuration specifies which LLM will generate the synthetic text fields. The example below defines a single text-generation model:

```
import data_designer.config as dd
```

```
model_configs = [
    dd.ModelConfig(
        provider="default/nvidia-build",
        model="nvidia/nemotron-3-nano-30b-a3b",
        alias="text",
    )
]


config_builder = dd.DataDesignerConfigBuilder(model_configs)
```

2.6 Step 3 – Build the Dataset Schema

As a worked example, a synthetic retail product review dataset was designed. Some columns use predefined sampler values, while the review text itself is generated by the LLM.

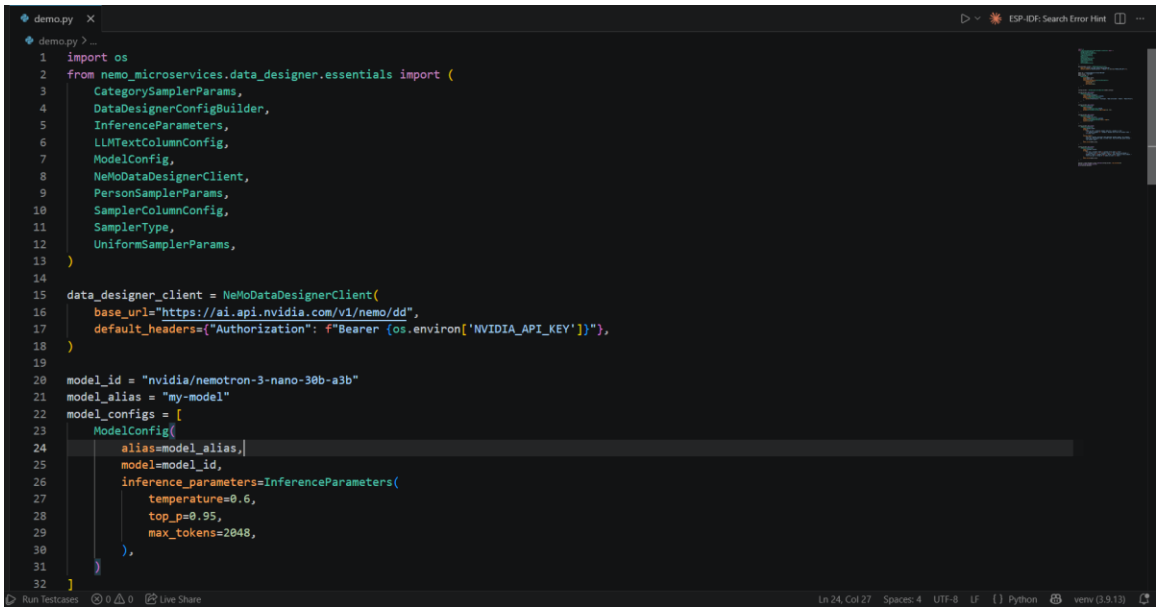
Column	Column Type	Description
product_category	Sampler (Category)	Randomly chosen from a fixed list of categories
customer_name	Sampler (Person)	Synthetic customer name
rating	Sampler (Integer, 1-5)	Randomly generated star rating
product_name	LLM Generated	Plausible product name for the category
review_text	LLM Generated	Review written to match the category and rating

```
config_builder.add_column(
    dd.SamplerColumnConfig(
        name="product_category",
        sampler_type=dd.SamplerType.CATEGORY,
        params=dd.CategorySamplerParams(
            values=["Electronics", "Home & Kitchen", "Sports", "Books"],
        ),
    )
```

```
)


config_builder.add_column(
    dd.SamplerColumnConfig(
        name="rating",
        sampler_type=dd.SamplerType.INTEGER,
        params=dd.IntegerSamplerParams(min_value=1, max_value=5),
    )
)


config_builder.add_column(
    dd.LLMTextColumnConfig(
        name="review_text",
        model_alias="text",
        prompt=(
            "Write a short, realistic customer review for a product in the "
            "{{ product_category }} category with a rating of {{ rating }} out
of 5."
        ),
    )
)
```



2.7 Step 4 – Preview the Dataset

Before generating a full dataset, Data Designer allows a small preview batch to be generated so the configuration can be checked and adjusted quickly.

```
preview = data_designer.preview(config_builder)

preview.display_sample_record()
```

2.8 Step 5 – Generate and Evaluate the Full Dataset

Once the preview looked realistic, a larger generation job was submitted and the results were downloaded as a dataset for evaluation.

```
job = data_designer.create(config_builder, num_records=50)

job.wait_until_done()



results = job.download_artifacts()

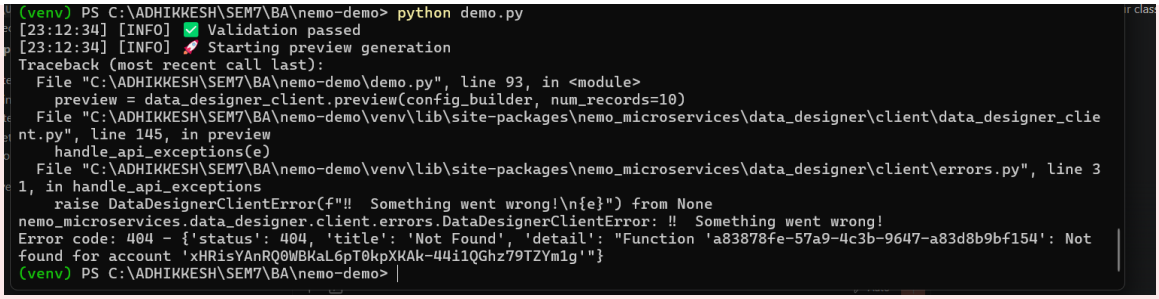
dataset = results.load_dataset()

print(dataset.head())



analysis = results.load_analysis()

analysis.to_report()
```



2.9 Seeding with Existing Data

Data Designer also supports seed datasets: instead of generating fully random records, an existing dataset (uploaded from HuggingFace or as an NMP Fileset) can be used as a starting point, so the synthetic records preserve the statistical patterns of the original data while adding new, non-identifying records.

2.10 Applications of Synthetic Data

Domain	Example Use
Healthcare	Synthetic patient records for model training
Banking	Fraud-detection training data without exposing real transactions
Retail	Simulated customer purchase and review data
Education	Practice datasets for student analytics exercises
Conversational AI / RAG	Synthetic question-answer pairs for evaluation

Comparative Summary

Aspect	Apify Web Scraper	NVIDIA NeMo Data Designer
Purpose	Extracts real data from live websites	Generates synthetic (artificial) data
Data source	Existing web pages	Large Language Model output, optionally seeded
Setup	Browser-based, no local install needed	Python SDK with API key
Output formats	JSON, CSV, Excel, XML, HTML	Dataset (pandas DataFrame), analysis report
Best suited for	Market research, price/review monitoring	Model training data, privacy-safe testing data

Advantages Observed

Apify

- Automates repetitive data collection with no local setup required.
- Handles both static and JavaScript-rendered (dynamic) websites.
- Exports directly to formats usable in Excel, Power BI, or Python.
- Configurable through a graphical interface without writing a full scraper from scratch.

NVIDIA NeMo Data Designer

- Generates large volumes of realistic data without manual labelling.
- Protects privacy by avoiding the use of real customer data.
- Preview step allows fast iteration before committing to a full generation run.
- Seed datasets allow synthetic data to preserve patterns from real data.

Limitations Observed

- Apify: modern websites with anti-scraping protections or dynamically changing HTML can cause fields to be missed or a run to be blocked entirely.
- NeMo Data Designer: the realism of generated data depends heavily on how well the schema and prompts are designed; poorly specified columns can produce implausible records.

Learning Outcomes

- Understood how CSS-selector-based scraping extracts structured data from static HTML.
- Learned how a Page Function combines jQuery with custom JavaScript logic to shape extracted records.

- Understood the difference between real-world data collection (scraping) and synthetic data generation (LLM-based).
- Learned how Data Designer separates schema configuration from execution, and why previewing before full generation matters.
- Identified practical Business Analytics applications for both tools, from price monitoring to privacy-safe model training data.

Conclusion

This activity provided hands-on exposure to two complementary approaches to building datasets for Business Analytics: extracting real data from the web using Apify, and generating synthetic data using NVIDIA NeMo Data Designer. The Apify demonstration showed how quickly a structured dataset can be built from a live website using a short Page Function, while also revealing the practical limits of simple CSS-selector scraping against protected or dynamic sites. The NeMo Data Designer demonstration showed how a dataset schema can be defined once and used to generate realistic, privacy-safe synthetic records at scale, with a preview step to validate quality before full-scale generation. Together, these tools illustrate two practical ways AI can accelerate the data-preparation stage of a Business Analytics workflow.

References

- Apify Web Scraper Actor: https://apify.com/apify/web-scraper
- Books to Scrape (practice site): https://books.toscrape.com/
- Quotes to Scrape (practice site): https://quotes.toscrape.com/
- NVIDIA NeMo Data Designer / Synthetic Data for Agentic AI: https://www.nvidia.com/en-us/use-cases/synthetic-data-generation-for-agentic-ai/